Access Modifiers, Encapsulation & Object Modeling

A Concept Introduction

What You'll Learn

#	Topic	The question it answers
1	Access Modifiers — private, default, protected, public	Who is actually allowed to see or touch this field or method?
2	Visibility Across Packages and Inheritance	Does being a subclass, or being in the same package, change what I can reach?
3	Encapsulation — Data Hiding	Why hide fields behind methods instead of just leaving them public?
4	The JavaBean Standard — Getters and Setters	What naming convention makes my class instantly recognisable to tools and frameworks?
5	Read-Only and Write-Only Properties	What if a value should be seen but never changed — or changed but never read back?
6	Immutable Objects	What happens if I remove every setter and never let a field change again?
7	final Fields and Encapsulation	How do final and private work together to build objects that truly protect themselves?

SECTION 1

Access Modifiers

private · default · protected · public

Back in Week 3, FeeAccount quietly used private for its fields and public for its constructor and methods, without much explanation of why. Java actually gives you four access levels, not two — and choosing the right one for every field and method is a real design decision, not a formality.

The Four Access Levels

Modifier	Keyword	Accessible From
private	private	Only code inside the exact same class.
default (package-private)	(no keyword at all)	Any class in the same package — nothing else.
protected	protected	Same package, plus any subclass, even in a different package.
public	public	Anywhere at all, with no restriction.

```
package srm.fees;

public class FeeAccount {
    private String regNo;              // only FeeAccount itself can touch this
    double totalFee;                   // default -- only classes inside srm.fees
    protected double amountPaid;       // srm.fees, plus any subclass anywhere
    public String accountType;         // any class, in any package

    public FeeAccount(String regNo, double totalFee) {
        this.regNo = regNo;
        this.totalFee = totalFee;
    }
}
```

WHY THIS MATTERS

Access modifiers are how you design an API on purpose, instead of by accident — every public member is a promise to every future caller that it will keep working exactly this way.

The narrower you can make something without breaking real code that needs it, the more freedom you keep to change your own implementation later without breaking anyone else's.

This is the mechanical foundation encapsulation is built on — you cannot hide a field behind a method unless the language actually enforces that hiding.

PAUSE & THINK

default access needs no keyword at all -- you simply write nothing. Why do you think Java chose "no keyword" to mean "package only", instead of making that the meaning of public, or requiring an explicit keyword for it?

Try This Code

```
package srm.fees;

public class FeeAccount {
    private String regNo;
```

```
    public double totalFee;

    public FeeAccount(String regNo, double totalFee) {
        this.regNo = regNo;
        this.totalFee = totalFee;
    }

    public String getRegNo() {
        return regNo;    // the only door into a private field
    }
}

// -- In a different file, same package --
package srm.fees;

public class FeeOfficeDemo {
    public static void main(String[] args) {
        FeeAccount acc = new FeeAccount("RA2311003010123", 200000);
        System.out.println(acc.totalFee);     // OK -- public
        System.out.println(acc.getRegNo());   // OK -- through a public method
        // System.out.println(acc.regNo);     -- COMPILE ERROR, private
    }
}
```

Output

```
200000.0
RA2311003010123
```

TRY BREAKING IT

Uncomment the acc.regNo line above and try to compile. Read the error carefully -- it will name the field and the exact reason: regNo has private access in FeeAccount.

WATCH OUT FOR

Assuming default access is the same as public just because it needs no keyword — it is actually one of the most restrictive levels, tighter than protected.

Reaching for public out of convenience during development, then never tightening it later — once other code depends on a public member, narrowing it is a breaking change.

QUICK RECAP

private: this class only. default: this package only. protected: this package + subclasses anywhere. public: everyone.

Choose the narrowest access level that still lets legitimate callers do their job.

SECTION 2

Visibility Rules Across Packages and Inheritance

How package boundaries and the extends relationship change what protected and default actually reach

Section 1 defined the four levels in the abstract. Once your classes start living in different packages, and subclassing across those package boundaries, the real behaviour of default and protected needs a second, more careful look.

The Full Visibility Matrix

Modifier	Same Class	Same Package	Subclass, Different Package	Unrelated Class, Different Package
private	Yes	No	No	No
default	Yes	Yes	No	No
protected	Yes	Yes	Yes	No
public	Yes	Yes	Yes	Yes

The one genuinely surprising row is protected: it reaches a subclass sitting in a completely different package, but only through inheritance -- an unrelated class in that same different package still cannot touch it.

```
// -- File: srm/fees/FeeAccount.java --
package srm.fees;

public class FeeAccount {
    protected double amountPaid;
    private String internalAuditCode;   // never leaves this class, ever

    protected void applyLatePenalty(double amount) {
        amountPaid -= amount;
    }
}

// -- File: srm/hostel/HostelFeeAccount.java --
package srm.hostel;

import srm.fees.FeeAccount;

public class HostelFeeAccount extends FeeAccount {
    public void chargeLateFine(double amount) {
        applyLatePenalty(amount);   // OK -- protected, reached through inheritance
        amountPaid += 0;            // OK -- protected field, same reasoning
        // System.out.println(internalAuditCode);  -- COMPILE ERROR, private
    }
}
```

WHY THIS MATTERS

protected exists specifically for this situation: exposing something to your own subclasses without handing it to the entire world through public.

Real Java projects are organised into packages precisely so that default and protected can express “internal to this module, or to code that genuinely extends it” — a middle ground public alone cannot express.

PAUSE & THINK

If HostelFeeAccount did NOT extend FeeAccount, but simply lived in a completely different package and tried to call applyLatePenalty() on a FeeAccount object it was handed, would that compile? Why or why not?

TRY BREAKING IT

Delete the extends FeeAccount relationship from HostelFeeAccount (keep the import) and try calling applyLatePenalty() directly on a FeeAccount object instead of through inheritance. Watch protected suddenly refuse you, even though you're still "related" through the import.

WATCH OUT FOR

Assuming protected behaves like public once you're in a subclass — it only opens the door through the inheritance relationship itself, not to the subclass's own unrelated helper classes.

Forgetting that default access is package-based, not folder-based — two classes in the same folder but declared in different package statements are not in the same package at all.

QUICK RECAP

default = same package only, full stop — inheritance across packages does not extend it.

protected = same package, plus subclasses anywhere — but only reachable through the inheritance relationship itself.

SECTION 3

Encapsulation — Data Hiding With Private Fields and Public Methods

The core discipline access modifiers exist to make possible

Encapsulation is the practice of keeping an object's fields private and exposing only the specific, controlled operations the outside world actually needs, through public methods. It's the single idea private and public were built to support.

Without Encapsulation

```
class FeeAccount {
    public double totalFee;
    public double amountPaid;
}

// -- Anywhere else in the codebase --
FeeAccount acc = new FeeAccount();
acc.amountPaid = -50000;   // nothing stops this. Nothing at all.
```

Every field being public means every single caller, everywhere in a large codebase, is individually trusted to keep amountPaid sensible. One careless line, anywhere, and the object's state stops making sense.

With Encapsulation

```
class FeeAccount {
    private double totalFee;
    private double amountPaid;

    public FeeAccount(double totalFee) {
        this.totalFee = totalFee;
    }

    public void pay(double amount) {
        if (amount <= 0) {
            System.out.println("Payment rejected: must be positive");
            return;
        }
        amountPaid += amount;
    }

    public double getDue() {
        return totalFee - amountPaid;
    }
}
```

Now there is exactly one door into amountPaid: the pay() method. Every rule about what counts as a valid payment lives in one place, and every caller, everywhere, is forced through it.

WHY THIS MATTERS

Encapsulation turns “all callers must remember the rules” into “the rules are enforced automatically, no matter who calls in.”

It also protects you as the class's author: you can completely rewrite how amountPaid is stored internally, and nothing outside the class needs to change, as long as pay() and getDue() keep behaving the same way.

This is exactly the same instinct behind private fields in every serious Java codebase, and behind why direct public field access is treated as a design smell in professional code review.

PAUSE & THINK

getDue() only reads state -- it never changes anything. pay() changes state, but only in a controlled way. Can you think of a method FeeAccount might need that does neither -- one that's genuinely read-only, and one that's genuinely a controlled write?

TRY BREAKING IT

Add a second public field, like public double totalFee, right alongside the encapsulated version. Now write a line elsewhere that sets totalFee directly to a negative number. Notice nothing stops you -- one public field is all it takes to punch a hole in otherwise-solid encapsulation.

WATCH OUT FOR

Encapsulating some fields but leaving even one field public — that single public field is enough to let bad state back in, no matter how careful the rest of the class is.

Writing a public setter that just does field = value with zero validation — technically “encapsulated,” but it protects nothing, since it's really just a public field wearing a method's clothing.

QUICK RECAP

Encapsulation = private fields + public methods that enforce the class's own rules.

One door in, one door out — every caller goes through the same validated path.

SECTION 4

The JavaBean Standard — Getters and Setters

A naming convention that turns your class into something tools, frameworks, and other developers can work with automatically

Encapsulation says “hide fields, expose methods.” The JavaBean standard says: name those methods this way, specifically, so that IDEs, frameworks (like Spring), and libraries that convert objects to JSON can all recognise your class's properties automatically, without you writing a single line of configuration.

The Naming Rules

Field	Getter	Setter
private String name;	public String getName()	public void setName(String name)
private int age;	public int getAge()	public void setAge(int age)
private boolean active;	public boolean isActive() — not getActive()	public void setActive(boolean active)

A full JavaBean also expects a public no-argument constructor, so frameworks can create an empty instance first and populate it property by property afterward.

```
public class SrmStudent {
    private String name;
    private int attendance;
    private boolean scholarshipHolder;

    public SrmStudent() { }   // public no-arg constructor -- part of the standard

    public String getName() { return name; }
    public void setName(String name) { this.name = name; }

    public int getAttendance() { return attendance; }
    public void setAttendance(int attendance) { this.attendance = attendance; }

    public boolean isScholarshipHolder() { return scholarshipHolder; }   // boolean -> is,
not get
    public void setScholarshipHolder(boolean scholarshipHolder) {
        this.scholarshipHolder = scholarshipHolder;
    }
}
```

WHY THIS MATTERS

Frameworks like Spring, and libraries like Jackson (which converts objects to and from JSON), rely entirely on this naming pattern to work automatically — break the convention, and they simply won't find your property.

Following the standard means anyone opening your class for the first time already knows exactly what getX() and setX() will do, without reading a single line of implementation.

PAUSE & THINK

isActive() for a boolean instead of getActive() looks like a small, arbitrary rule. What real problem might it be quietly solving for a framework scanning your class through reflection?

TRY BREAKING IT

Rename isScholarshipHolder() to getScholarshipHolder() and imagine (or actually test with a JSON library if you have one available) what happens when a JavaBean-aware tool looks for a boolean property named scholarshipHolder -- some tools will simply fail to find it.

WATCH OUT FOR

Writing a getter or setter that does real, unexpected work beyond returning or assigning the field — callers expect getX() to be cheap and side-effect-free, since that's the whole convention's unspoken promise.

Forgetting the public no-argument constructor — a class with only a parameterized constructor is not a full JavaBean, even if every getter and setter is named perfectly.

QUICK RECAP

getX() / setX() for ordinary fields; isX() (not getX()) for boolean fields.

A full JavaBean also needs a public no-argument constructor.

SECTION 5

Read-Only and Write-Only Properties

Not every field needs both halves of the getter/setter pair

A JavaBean-style property doesn't have to come as a matching getter-and-setter pair. Providing only a getter creates a read-only property; providing only a setter creates a write-only one — and both are legitimate, deliberate design choices.

Read-Only — Getter, No Setter

A value that's fixed once the object is built, and should only ever be read afterward, gets a getter and nothing else.

```
public class SrmStudent {
    private final String regNo;   // set once, in the constructor
    private String name;

    public SrmStudent(String regNo, String name) {
        this.regNo = regNo;
        this.name = name;
    }

    public String getRegNo() { return regNo; }   // getter only -- read-only property
    public String getName() { return name; }
    public void setName(String name) { this.name = name; }   // name can still change
}
```

Write-Only — Setter, No Getter

A value that must be provided but should never be read back out — a password being the classic example — gets a setter and deliberately no matching getter.

```
public class PortalAccount {
    private String username;
    private String passwordHash;

    public void setPassword(String plainPassword) {
        this.passwordHash = hash(plainPassword);   // stored, but...
    }
    // Deliberately no getPassword() -- nothing should ever read it back out

    public String getUsername() { return username; }
    public void setUsername(String username) { this.username = username; }

    private String hash(String input) { return "hashed:" + input; }
}
```

WHY THIS MATTERS

Read-only properties let you expose information freely while still protecting exactly how or when it can change — regNo is safe to hand out everywhere, precisely because nothing can hand a new value back in.

Write-only properties are a real security pattern, not just a style choice — a class that genuinely cannot return a password can't leak it later either, even by accident.

PAUSE & THINK

getRegNo() has no matching setRegNo(). Is SrmStudent immutable? What would you still need to check to be sure?

TRY BREAKING IT

Add a getPassword() method to PortalAccount that returns passwordHash, breaking the write-only design. Nothing will stop you -- Java enforces access modifiers, not design intent. Think about what kind of code review comment this should draw.

WATCH OUT FOR

Assuming a getter-only field is automatically immutable — it only means this one field can't be reassigned through a setter; other methods could still mutate the object some other way.

Adding a getter “just in case it's useful later” on a field that was deliberately write-only — that one convenience getter quietly undoes the entire safeguard.

QUICK RECAP

Read-only property: getter, no setter. Write-only property: setter, no getter.

Both are deliberate design decisions enforced by simply not writing the other half of the pair.

SECTION 6

Immutable Objects

What happens when every single field is read-only, all at once

Take the read-only property pattern and apply it to every field in a class, and you get an immutable object: once constructed, its state can never change again, for the rest of its life. You've already met one — String itself is immutable.

```
public final class EnrollmentRecord {
    private final String regNo;
    private final String course;
    private final int enrollmentYear;

    public EnrollmentRecord(String regNo, String course, int enrollmentYear) {
        this.regNo = regNo;
        this.course = course;
        this.enrollmentYear = enrollmentYear;
    }

    public String getRegNo() { return regNo; }
    public String getCourse() { return course; }
    public int getEnrollmentYear() { return enrollmentYear; }
    // No setters anywhere. Not one.

    // "Changing" an immutable object means building a NEW one instead
    public EnrollmentRecord withCourse(String newCourse) {
        return new EnrollmentRecord(this.regNo, newCourse, this.enrollmentYear);
    }
}
```

Notice withCourse() doesn't modify the existing object at all — it builds and returns a brand-new one, leaving the original completely untouched. This is exactly the pattern String.toUpperCase() and String.substring() already follow.

WHY THIS MATTERS

An immutable object can be safely shared everywhere — handed to another method, stored in a collection, passed across threads — with zero risk that something else quietly changes it out from under you.

It also makes reasoning about code dramatically simpler: if you're holding an EnrollmentRecord, you know with certainty its regNo will be the same value tomorrow as it is right now.

This is precisely why Java itself made String immutable — file paths, class names, and map keys all depend on being able to trust that a string won't silently change after being checked.

PAUSE & THINK

final class, final fields, no setters -- that's three separate safeguards working together in EnrollmentRecord. Which one of the three, if you removed it, would break immutability the fastest?

TRY BREAKING IT

Remove just the final keyword from the class declaration (keep everything else). EnrollmentRecord itself still behaves identically -- but now think about what a subclass could theoretically add. Does removing final from the class alone actually break immutability of existing EnrollmentRecord objects?

WATCH OUT FOR

Declaring fields final but forgetting that a final reference to a mutable object (like an array or a List) still lets that object's contents change — final protects the reference, not automatically the thing it points to.

Writing a “wither” method like withCourse() that accidentally mutates this instead of building a new object — that one bug quietly turns an immutable class into a mutable one with extra steps.

QUICK RECAP

Immutable = every field final, no setters anywhere, class itself often marked final.

“Changing” an immutable object means constructing and returning a new one — the original is never touched.

SECTION 7

final Fields and Encapsulation

Where final and private meet -- and the one gap even they don't close on their own

private controls who can see a field. final controls whether it can ever be reassigned. Used together, they're the strongest protection a single field can get in Java — with one remaining subtlety worth understanding clearly.

The Gap: final Protects the Reference, Not Always the Contents

A final field holding a primitive (like a String's own immutability, or an int) is genuinely locked down completely. But a final field holding a mutable object — an array, or an ArrayList — only guarantees the field can't be pointed at a different object. The object it already points to can still be changed from the outside, unless you defend against that separately.

```
public class CourseRoster {
    private final String courseCode;
    private final int[] scores;   // final -- but this does NOT freeze the array's contents

    public CourseRoster(String courseCode, int[] scores) {
        this.courseCode = courseCode;
        this.scores = scores;         // DANGER: stores the caller's own array directly
    }

    public int[] getScores() {
        return scores;                // DANGER: hands out the real internal array
    }
}

// -- Elsewhere --
int[] marks = {90, 85, 76};
CourseRoster roster = new CourseRoster("21CSC201J", marks);
marks[0] = 0;                         // the roster's "private final" data just changed too!
roster.getScores()[1] = 0;            // and this changes it from OUTSIDE the class entirely
```

Closing the Gap: Defensive Copying

```
public class CourseRoster {
    private final String courseCode;
    private final int[] scores;

    public CourseRoster(String courseCode, int[] scores) {
        this.courseCode = courseCode;
        this.scores = scores.clone();      // copy IN -- caller's array can no longer reach
ours
    }

    public int[] getScores() {
        return scores.clone();             // copy OUT -- caller can't reach our real array
either
    }
}
```

WHY THIS MATTERS

final alone is not the full story for any field whose type is itself mutable — real encapsulation for arrays, collections, and other mutable objects needs a defensive copy on the way in and, often, on the way out.

This exact gap is a genuinely common source of bugs in real codebases: a class that looks perfectly encapsulated on paper, with every field private final, that can still be silently mutated from the outside through a shared array or list reference.

PAUSE & THINK

private final int[] scores looks airtight at a glance. Once you've seen the defensive-copy version, what's the one-sentence rule you'd give a teammate for when final alone is genuinely enough, versus when it needs a .clone() alongside it?

TRY BREAKING IT

Remove just the .clone() from the constructor (keep it in getScores()). Then externally mutate the array you passed into the constructor, the way marks[0] = 0; does above. Confirm for yourself that the object's internal state changed anyway -- despite scores being declared final the entire time.

WATCH OUT FOR

Assuming final on a field is a complete guarantee of immutability — it only guarantees the reference can't be reassigned, never automatically that the object it refers to can't be mutated.

Handing out a live reference to an internal array or ArrayList through a getter — even a private final field becomes freely mutable from outside the class the moment you do this.

QUICK RECAP

final locks the reference: you can never reassign the field to point somewhere else.

final does NOT automatically lock the contents of a mutable object the field points to.

Defensive copying (in the constructor, and again in any getter) is what closes that gap for fields like arrays and collections.

WRAP-UP

Object Modeling: Putting It All Together

Access modifiers, encapsulation, JavaBeans, and immutability aren't separate topics — they're the same skill, seen from different angles

Every idea in this document answers one underlying question: when you model a real thing as a Java object, exactly how much of its internal state and behaviour should the outside world be allowed to touch? A well-modeled class is deliberate about every single field: is it hidden entirely, exposed read-only, exposed read-write through a validated setter, or fixed forever the moment the object is built?

```
public final class SrmStudentProfile {
    private final String regNo;        // SECTION 6/7: final -- fixed forever, set once
    private final String name;         // final -- immutable identity
    private double attendance;         // SECTION 3: private -- hidden, but allowed to
change

    public SrmStudentProfile(String regNo, String name, double attendance) {
        this.regNo = regNo;
        this.name = name;
        this.attendance = attendance;
    }

    public String getRegNo() { return regNo; }     // SECTION 5: read-only property
    public String getName() { return name; }        // SECTION 5: read-only property

    public double getAttendance() { return attendance; }         // SECTION 4: JavaBean pair
    public void setAttendance(double attendance) {                // SECTION 3: validated,
not raw
        if (attendance < 0 || attendance > 100) {
            System.out.println("Rejected: attendance must be 0-100");
            return;
        }
        this.attendance = attendance;
    }
}
```

One class, four deliberate decisions: regNo and name are permanently read-only because a student's identity shouldn't drift; attendance is read-write because it genuinely changes week to week, but only through a setter that enforces the rule that attendance stays between 0 and 100. Nothing here is accidental — every access level and every final was chosen on purpose.

QUICK RECAP

Object modeling is the design skill; access modifiers, encapsulation, JavaBeans, and immutability are the four tools you use to express it in code.

For every field, ask: does this need to change after construction? Should the outside world read it? Should the outside world write it, and if so, under what rule?

Cheat Sheet

Topic	Remember This
Access Modifiers	private: this class only. default: this package only. protected: package + subclasses anywhere. public: everyone.
Visibility Across Packages	protected reaches a subclass in another package only through the inheritance relationship itself, never sideways to unrelated classes.
Encapsulation	Private fields, public methods that enforce the class's own rules. One door in, one door out -- every caller goes through it.
JavaBean Standard	getX() / setX() for normal fields, isX() for booleans, plus a public no-argument constructor.
Read-Only / Write-Only	Getter with no setter = read-only. Setter with no getter = write-only. Both are deliberate, not accidental.
Immutable Objects	Every field final, no setters at all. “Changing” means building and returning a new object.
final Fields	Locks the reference, not automatically the contents of a mutable object it points to -- defensive copying closes that gap.

Test Yourself

Answer from memory first. Whatever you cannot answer tells you exactly which section to reread.

1. What is the one situation where protected actually reaches code in a different package -- and why does it not reach every class in that package?

2. Give a concrete example of a bug that public fields alone would allow, which encapsulation prevents.

3. Why does a boolean JavaBean property use isX() instead of getX()?

4. Describe a real field that should be a read-only property, and a real field that should be write-only.

5. What does it mean to "change" an immutable object, if none of its fields can actually be reassigned?

6. A class has a private final int[] field with no defensive copying. Describe exactly how an outside caller could still mutate it.

7. Why is a public no-argument constructor part of the JavaBean standard, even for a class whose fields are all mandatory in spirit?

8. In the SrmStudentProfile example, why are regNo and name final while attendance is not -- what's the underlying design reason, not just "because the code says so"?